Programa:	Advanced Tracks Cleaner 1.7
Protección:	Time-Trial x días
Objetivos:	Evitar que expire
URL descarga:	www.winsoul.com
Dificultad:	Muy fácil
Herramientas:	OllyDbg 1.10 + PEiD 0.93 + PEditor + UEdit32 + ResHacker
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinos] Fecha: 14-10-2005

Hola a todos..!!

Hoy vamos a mirar de solventar la protección del programa "Advanced Tracks Cleaner 1.7" a decir verdad todavía no sé para que diablos se necesita un programa así ¿¿?? bueno la verdad que sí que lo se jeje.. y vosotros también lo sabéis, sin perder tiempo nos vamos de cabeza al PEiD y que nos analice el archivo a ver que dice:

Borland Delphi 6.0 - 7.0

Pues nada es un Delphi, ni comprimido ni nada de nada así que trabajaremos directamente bajo Ollydbg 1.10, venga carguemos el exe en Olly y demos RUN, al poco carga el programa sin problemas, tampoco tiene antidebug, además lo único que vemos que no nos gusta en el programa es un UNREGISTERED en la barra de título, seamos un poco más agresivos y adelantemos el reloj un año, vamos a expirar el programa ahora, damos RUN y:

nos invita a salir del programa por la puerta grande el muy condenado, ni si quiera nos da la opción de registrarnos mediante una combinación nombre y serial como en la mayoría de los casos, este se deberá registrar mediante algún archivo .key o algo así me imagino, eso sí de momento no lo sabemos. Lo que sí sabemos es que a partir de cierto tiempo el programa no nos deja entrar más sacando la nag que vemos arriba, usaremos el método del BP en DestroyWindow para encontrar el CALL que se encarga de crear y mostrar la ventana, hacemos una cosa, reiniciamos Olly con Ctrl+F2 y damos RUN, en el momento que aparezca la nag en pantalla ponemos el BP:

Command : bp DestroyWindow

Y pulsamos "Exit" para cerrar la ventana, al momento salta Olly se ha llamado a la api:

```
77D1A185   B8 63110000   MOV EAX,1163
77D1A18A   BA 0003FE7F   MOV EDX,7FFE0300
77D1A18F   FFD2          CALL EDX
77D1A191   C2 0400       RETN 4
77D1A194   8B75 0C       MOV ESI,DWORD PTR SS:[EBP+C]
77D1A197   85F6          TEST ESI,ESI
77D1A199   74 0C         JE SHORT user32.77D1A1A7
```

Ah! no olvidar de quitar el BP cuando haya hecho efecto para que no moleste

En este momento estamos en la librería User32 lejos del código del ejecutable que estamos estudiando, lo que haremos será ir retrocediendo con Ctrl+F9 hasta encontrar el CALL que crea la ventana como dijimos antes, darse cuenta que estamos ante el CALL correcto es cuestión de fijarse en el entorno de donde está, mirar los saltos condicionales si hubiera, mensajes, etc., os pongo un ejemplo:

```
00474458  .   E8 CF280000  CALL Advanced.00476D2C
0047445D  .   8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
00474460  .   E8 77E1FEFF  CALL Advanced.004625DC          CALL que controla el salto
00474465  .   84C0         TEST AL,AL
00474467  .v  74 0B        JE SHORT Advanced.00474474      salto que evita la nag
00474469  .   8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
0047446C  .   8B10         MOV EDX,DWORD PTR DS:[EAX]
0047446E  .   FF92 AC00000 CALL DWORD PTR DS:[EDX+AC]      call_nag
00474474  >   8B55 FC      MOV EDX,DWORD PTR SS:[EBP-4]    Aqui salimos
00474477  .   A1 F44B4A00  MOV EAX,DWORD PTR DS:[4A4BF4]
0047447C  .   E8 BF500000  CALL Advanced.00479540
```

Esto es lo que os decía, hay zonas que nos pueden confundir pero en este caso se ve que aunque el salto se ejecute o no se ejecute siempre se pasará por :474474 así que no nos vale de mucho, seguimos traceanto con Ctrl+F9 y llegamos a esta otra parte:

```
004A017B  .   E8 2448F6FF  CALL Advanced.004049A4
004A0180  .   8B85 8CD7FFF MOV EAX,DWORD PTR SS:[EBP-2874]
004A0186  .   E8 49F4FFFF  CALL Advanced.0049F5D4
004A018B  .   3C 01        CMP AL,1
004A018D  .v  74 0F        JE SHORT Advanced.004A019E      salto malo.!!
004A018F  .   B8 07000000  MOV EAX,7
004A0194  .   E8 A7E6FDFF  CALL Advanced.0047E840
004A0199  .   83F8 07      CMP EAX,7
004A019C  .v  7C 3A        JL SHORT Advanced.004A01D8      salto BUENO..!!
004A019E  >   33C0         XOR EAX,EAX
004A01A0  .   55           PUSH EBP
004A01A1  .   68 C0014A00  PUSH Advanced.004A01C0
004A01A6  .   64:FF30      PUSH DWORD PTR FS:[EAX]
004A01A9  .   64:8920      MOV DWORD PTR FS:[EAX],ESP
004A01AC  .   B8 80034A00  MOV EAX,Advanced.004A0380       ASCII "suxx"
004A01B1  .   E8 228DF6FF  CALL Advanced.00408ED8
004A01B6  .   33C0         XOR EAX,EAX
004A01B8  .   5A           POP EDX
004A01B9  .   59           POP ECX
004A01BA  .   59           POP ECX
004A01BB  .   64:8910      MOV DWORD PTR FS:[EAX],EDX
004A01BE  .v  EB 18        JMP SHORT Advanced.004A01D8
004A01C0  .^  E9 E33BF6FF  JMP Advanced.00403DA8
004A01C5  .   8B15 F8A8480 MOV EDX,DWORD PTR DS:[48A8F8]   Advanced.0048A944
004A01CB  .   8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
004A01CE  .   E8 C9F2FFFF  CALL Advanced.0049F49C          call_nag
004A01D3  .   E8 383FF6FF  CALL Advanced.00404110
004A01D8  >   33C0         XOR EAX,EAX
004A01DA  .   5A           POP EDX
004A01DB  .   59           POP ECX
004A01DC  .   59           POP ECX
```

El sitio donde puse los BP's son los lugares interesantes de la rutina que tenemos delante, dejo anotado el CALL que nos saca la molesta ventana y por encima vemos dos saltos que tienen su papel, el primero de ellos (JE) si se produce nos damos de morros con ella, si no se produce dependerá de lo que haga el segundo (JL), lo que nos interesa es que no salte en el primero y que sí lo haga en el segundo y evitaríamos el CALL maligno, lo haremos fácil

```
004A0186   .   E8 49F4FFFF   CALL Advanced.0049F5D4    => Retornará AL==0
004A0194   .   E8 A7E6FDFF   CALL Advanced.0047E840    => Retornará EAX<7
```

Pues lo dicho no nos vamos a complicar la vida demasiado, ponemos un BP en cada uno de los dos CALL y una vez allí parados entramos a ver donde toma el valor de retorno AL/EAX, para el primer caso tenemos:

```
0049F655  L.  E8 8250F6FF  CALL Advanced.004046DC
0049F65A  L.  C3           RETN
0049F65B  .^  E9 FC49F6FF  JMP Advanced.0040405C
0049F660  .^  EB F0        JMP SHORT Advanced.0049F652
0049F662      8BC3         MOV EAX,EBX
0049F664  .   5B           POP EBX
0049F665  .   8BE5         MOV ESP,EBP
0049F667  .   5D           POP EBP
0049F668  .   C3           RETN
```

EAX toma su valor

Ahí vemos el primer punto, para que quede AL==0 lo tenemos fácil lo dejamos así:

```
0049F655  L.  E8 8250F6FF  CALL Advanced.004046DC
0049F65A  L.  C3           RETN
0049F65B  .^  E9 FC49F6FF  JMP Advanced.0040405C
0049F660  .^  EB F0        JMP SHORT Advanced.0049F652
0049F662      B0 00        MOV AL,0
0049F664  .   5B           POP EBX
0049F665  .   8BE5         MOV ESP,EBP
0049F667  .   5D           POP EBP
0049F668  .   C3           RETN
```

Le damos el valor directamente..!!

Así lo dejamos y vayamos hacia el otro CALL que está en :4A0194 donde hacemos los mismo, nos paramos en él y entramos buscando el lugar donde EAX toma su valor de retorno:

```
0047EB8B  .  BA 10000000  MOV EDX,10
0047EB90  .  E8 6B5BF8FF  CALL Advanced.00404700
0047EB95  .  C3           RETN
0047EB96  .^ E9 C154F8FF  JMP Advanced.0040405C
0047EB9B  .^ EB DB        JMP SHORT Advanced.0047EB78
0047EB9D     8BC6         MOV EAX,ESI
0047EB9F  .  5F           POP EDI
0047EBA0  .  5E           POP ESI
0047EBA1  .  5B           POP EBX
0047EBA2  .  8BE5         MOV ESP,EBP
0047EBA4  .  5D           POP EBP
0047EBA5  .  C3           RETN
```

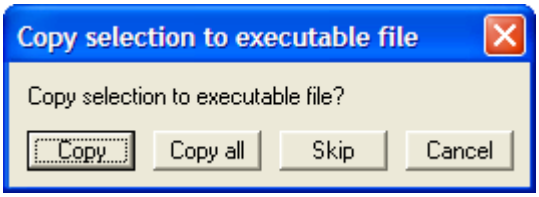
EAX toma su valor de retorno..!!

Como antes volvemos a encontrarnos con el lugar donde se modifica EAX por última vez antes del RET de la función en cuestión, este valor será el definitivo puesto que como se puede apreciar no se modificará más y saldrá, entonces como necesitamos un valor inferior a 7 podríamos probar a poner algo así:

```
0047EB8B  .  BA 10000000  MOV EDX,10
0047EB90  .  E8 6B5BF8FF  CALL Advanced.00404700
0047EB95  .  C3           RETN
0047EB96  .^ E9 C154F8FF  JMP Advanced.0040405C
0047EB9B  .^ EB DB        JMP SHORT Advanced.0047EB78
0047EB9D     33C0         XOR EAX,EAX
0047EB9F  .  5F           POP EDI
0047EBA0  .  5E           POP ESI
0047EBA1  .  5B           POP EBX
0047EBA2  .  8BE5         MOV ESP,EBP
0047EBA4  .  5D           POP EBP
0047EBA5  .  C3           RETN
```

Se lo cambiamos..!!

Y lo ponemos a 0, veremos si funciona de esta manera o por el contrario deberemos buscar alguna otra solución, de momento salvamos los cambios: Botón derecho sobre la ventana de código y elegimos "Copy to executable->All modifications" ya que tenemos más de un cambio (en concreto dos) y sale esta ventanita:



Elegimos "Copy All" y nos abre otra pantalla donde picamos otra vez con el botón derecho y elegimos "Save File" le ponemos un nombre y ya está hecho, probamos el archivo y.. de momento funciona, no del todo porque pone UNREGISTERED todavía pero ya están las funciones habilitadas otra vez ;-) además no vemos la nag y del límite de tiempo nos podemos olvidar, pues eso nos queda evitar algunas cadenas molestas como la citada anteriormente, veámosla:

A esto me refería, esta cadena no nos la debe insertar porque queda bastante fea, he buscado en Olly y la cadena no aparace, pero sí en UltraEdit ;-)

```
000bd5d0h: 63 6B 73 20 43 6C 65 61 6E 65 72 20 2D 20 55 4E ; cks Cleaner - UN
000bd5e0h: 52 45 47 49 53 54 45 52 45 44 20 56 45 52 53 49 ; REGISTERED VERSI
000bd5f0h: 4F 4E 0C 43 6C 69 65 6E 74 48 65 69 67 68 74 03 ; ON.ClientHeight.
000bd600h: 7F 01 0B 43 6C 69 65 6E 74 57 69 64 74 68 03 DF ; ▯..ClientWidth.ß
```

Se ve que está a partir del offset 0xbd5de pero curiosamente en Olly no aparece la cadena en esta dirección (más el imagebase claro) que sería 0BD5DE+400000=4BD5DE, si vamos allí vemos esto.

```
Address   Hex dump                                          ASCII
004BD5DE  00 DF 04 21 27 FF 1B C7 EB FF 1E DD FE FF 1E E4  .■♦!' +ãÛ ▲|■ ▲ä
004BD5EE  FF FF 21 EB FF FF 29 D6 E3 FF 07 23 25 FF 00 00    !Û  )íÛ ·#% ..
004BD5FE  00 FF 00 00 00 FF 35 D2 D4 FF 2D B6 B8 FF 19 70  . ... 5ÉÊ -Ã0 ↓p
004BD60E  72 FF 3B E6 EC FF 22 83 88 FF 00 00 00 FF 00 00  r ;µÿ "âê ... ..
```

Ya lo ven, si vamos a la ventana de Ejecutables nos dice que el imagebase es efectivamente 400000 así que no lo entiendo ¿? si alguien conoce el motivo agradecería que me lo explicara nunca está de más aprender cosas nuevas no creéis? desde aquí digo que será bienvenida cualquier explicación sobre el tema...

Como no podía parar de pensar en este tema comiéndome la cabeza el porqué podía ser seguí mirando y al final encontré una similitud en los bytes, es decir encontré el lugar exacto desde donde comienzan los bytes de la primera sección, en el disco son a partir del offset 0x400 y en memoria a partir del offset x1000 como se ve en la imagen:

En disco:

```
00000400h: 04 10 40 00 03 07 42 6F 6F 6C 65 61 6E 01 00 00 ; ..@...Boolean...
00000410h: 00 00 01 00 00 00 00 10 40 00 05 46 61 6C 73 65 ; ........@..False
00000420h: 04 54 72 75 65 8D 40 00 2C 10 40 00 02 04 43 68 ; .True▯@.,.@...Ch
00000430h: 61 72 01 00 00 00 00 FF 00 00 00 90 40 10 40 00 ; ar.....ÿ...▯@.@.
00000440h: 01 08 53 6D 61 6C 6C 69 6E 74 02 00 80 FF FF FF ; ..Smallint..€ÿÿÿ
00000450h: 7F 00 00 90 58 10 40 00 01 07 49 6E 74 65 67 65 ; ▯..▯X.@...Intege
```

Vemos los bytes "04 10 40 00 03 ..." bien miremos en Olly y encontramos:

```
Address   Hex dump                                          ASCII
00401000  04 10 40 00 03 07 42 6F 6F 6C 65 61 6E 01 00 00  ♦►@.♥•Boolean☺..
00401010  00 00 01 00 00 00 00 10 40 00 05 46 61 6C 73 65  ..☺....►@.♣False
00401020  04 54 72 75 65 8D 40 00 2C 10 40 00 02 04 43 68  ♦True▯@.,►@.☻♦Ch
00401030  61 72 01 00 00 00 00 FF 00 00 00 90 40 10 40 00  ar☺.... ...É@►@.
00401040  01 08 53 6D 61 6C 6C 69 6E 74 02 00 80 FF FF FF  ☺◘Smallint☻.Ç
```

jejeje los mismos bytes pero a partir de otra posición, pero no nos quedemos aquí, averigüemos el porqué de esto, abramos el exe en PEditor, ProcDump, etc., igual da y miremos las secciones que trae:

Section	Virtual Size	Virtual Offset	Raw Size	Raw Offset	Characteristics
CODE	000A0EDC	00001000	000A1000	00000400	60000020
DATA	00001834	000A2000	00001A00	000A1400	C0000040
BSS	00000DF1	000A4000	00000000	000A2E00	C0000000
.idata	0000293C	000A5000	00002A00	000A2E00	C0000040

Allí están marcados ambos datos, se ve como la primera sección (PE header) estará en el disco (raw=crudo) a partir del offset 0x400h y como en memoria (Virtual Offset) cambia al offset 0x1000h, teniendo esto presente podemos buscar la cadena UNREGISTERED otra vez si queremos ya que ahora sólo calculando la diferencia nos saldrá el offset real donde estará la cadena (o al menos debería de estar, jeje)

Virtual = Disco + 600h (para la primera sección)

Offset_disco cadena: 0xbd5deh
Offset_virtual: 0xbd5deh + 600h = 0bdbdeh

Debemos pensar que la cadena no está en la primera sección por la dirección que nos da :4BD4BD, veamos donde estará desde el propio Olly en la ventana de memoria:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
003DC000	00001000	SHLWAPI	.data	data	Imag	R	RWE	
003DD000	00002000	SHLWAPI	.rsrc	resources	Imag	R	RWE	
003DF000	00005000	SHLWAPI	.reloc	relocations	Imag	R	RWE	
003F0000	00008000				Priv	RW	RW	
00400000	00001000	Advanced		PE header	Imag	R	RWE	
00401000	000A1000	Advanced	CODE	code	Imag	R	RWE	
004A2000	00002000	Advanced	DATA	data	Imag	R	RWE	
004A4000	00001000	Advanced	BSS		Imag	R	RWE	
004A5000	00003000	Advanced	.idata	imports	Imag	R	RWE	
004A8000	00001000	Advanced	.tls		Imag	R	RWE	
004A9000	00001000	Advanced	.rdata		Imag	R	RWE	
004AA000	0000C000	Advanced	.reloc	relocations	Imag	R	RWE	
004B6000	0006C000	Advanced	.rsrc	resources	Imag	R	RWE	
00530000	00006000				Map	R E	R E	
005F0000	00002000				Map	R E	R E	
00600000	00103000				Map	R	R	
00710000	000D0000				Map	R E	R E	

Vemos que cae dentro de la sección .rsrc (la de recursos) entonces nos iremos con un desplazamiento +600 sino algo superior, veamos en PEditor cuanto es:

Section	Virtual Size	Virtual Offset	Raw Size	Raw Offset	Characteristics
.rdata	00000018	000A9000	00000200	000A5800	50000040
.reloc	0000BB5C	000AA000	0000BC00	000A5A00	50000040
.rsrc	0006B600	000B6000	0006B600	000B1600	50000040

La diferencia sale como ya sabemos restándole al Virtual_Offset el valor de Raw_Offset:

0B6000 - 0B1600 = 4A00h

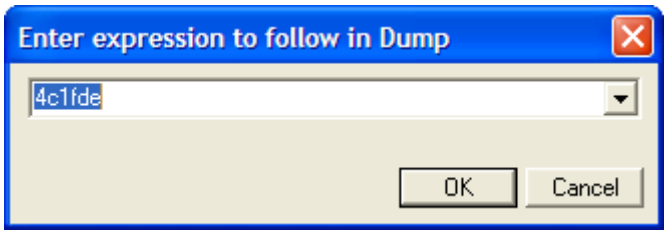
Eso es trabajamos con un desfase de 4A00h en el offset del disco a la memoria, sabemos que en el disco está en 0BD5DEh en memoria sería:

0BD5DE + 4A00 = C1FDEh (ojo este es el offset)

para obtener la dirección real le debemos añadir la imagebase que es 400000h y entonces:

C1FDE + 400000 = 4C1FDEh

Si ahora vamos a esa dirección con Ctr+G desde la ventana de dump (datos):



Lógicamente encontramos lo que venimos buscando ;-)

```
Address   Hex dump                                          ASCII
004C1FDE  55 4E 52 45 47 49 53 54 45 52 45 44 20 56 45 52  UNREGISTERED VER
004C1FEE  53 49 4F 4E 0C 43 6C 69 65 6E 74 48 65 69 67 68  SION.ClientHeigh
004C1FFE  74 03 7F 01 0B 43 6C 69 65 6E 74 57 69 64 74 68  t♥⌂0∂ClientWidth
004C200E  03 DF 01 05 43 6F 6C 6F 72 07 09 63 6C 42 74 6E  ♥▀0♣Color•.clBtn
004C201E  46 61 63 65 0C 46 6F 6E 74 2E 43 68 61 72 73 65  Face.Font.Charse
004C202E  74 07 0F 52 55 53 53 49 41 4E 5F 43 48 41 52 53  t•☼RUSSIAN CHARS
```

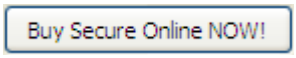
Allí está sana y salva la dichosa cadenita, no sé porqué los métodos de búsqueda fallan, incluso lo busqué en formato Unicode al ser la sección de recursos pero tampoco ¿? pero bueno lo importante es que tenemos la cadena localizada. He de decir que a pesar de todo intenté seguirle la pista y fue difícil, primero se crea se propio espacio de memoria donde copia la cadena, después la vuelve a copiar a otra zona temporal de dirección muy pareja, consigo encontrar algunos saltos donde evito que la pinte pero también evita que se escriban el resto de cadenas del programa quedando vacío, la solución más sencilla sería modificar el primer carácter por 00 (NULL) o byte nulo que le haga pensar que ya la ha escrito cuando en realidad ni ha comenzado, esta es una técnica muy usada por todos nosotros y aparece en muuuuchos tutes, sabiendo la dirección real de la cadena en memoria (y en disco) la podemos modificar, dejamos esto:

```
Address   Hex dump                                          ASCII
004C1FDE  00 4E 52 45 47 49 53 54 45 52 45 44 20 56 45 52  .NREGISTERED VER
004C1FEE  53 49 4F 4E 0C 43 6C 69 65 6E 74 48 65 69 67 68  SION.ClientHeigh
004C1FFE  74 03 7F 01 0B 43 6C 69 65 6E 74 57 69 64 74 68  t♥⌂0∂ClientWidth
004C200E  03 DF 01 05 43 6F 6C 6F 72 07 09 63 6C 42 74 6E  ♥▀0♣Color•.clBtn
```

Y salvamos los cambios como sabemos, lo arrancamos de nuevo y...

jeje pues faltó un poco, lo retocamos dejando los 00's sustituyendo al char "-" y no habrá más problemas, incluso el que quiera puede poner su nombre. Con este cambio sólo nos queda hacer lo mismo en la ventanita del About, como es el mismo trabajo y no aprenderemos nada nuevo puedo decir que se encuentra a partir del offset 0xf2e23 allí sí estaría bien poner nuestro nombre ;-)

Como último detalle podríamos eliminar el botón "Buy Now" del About con Resource Hacker, veamos primero el botoncito a ver que texto contiene:



Buscaremos "Buy Secure Online NOW!" en el ResHacker a ver que dice:

```
object Button2: TButton
  Left = 128
  Top = 302
  Width = 145
  Height = 25
  Caption = 'Buy Secure Online NOW!'
  TabOrder = 2
  OnClick = Button2Click
end
```

Esto es lo que nos dio, jeje suficiente le cambiaremos cualquier valor para que no se muestre y ya está, yo le hice esto:

```
end
object Button2: TButton
  Left = 128
  Top = 302
  Width = 0
  Height = 0
  Caption = 'Buy Secure Online NOW!'
  TabOrder = 2
  OnClick = Button2Click
end
```

Entonces en realidad lo pone pero de anchura=altura=0 ;-) o sea que nos hay botón:

Ahora vemos esto.

El trabajo lo hicimos compañeros, esperemos que no falte nada pues nunca se sabe en estos temas, piensas que está todo hecho y luego falla por todos lados, eso se verá con el tiempo si nadie se queja es que no hay problema.

Sólo me queda deciros que espero que hayáis pasado un rato entretenido leyendo el tute y hayáis aprendido aunque sólo sea una cosita. Para cualquier cosa estaré en:

syxe00@yahoo.es

Hasta la próxima ;-DDD